

TipTop Technologies

Monolith Handover Document

Intern: Lucas Li

Role: AI Platforms Development & Marketing Intern

Project: Monolith Fitness App

Date: June - August 2026

1. Project Overview

During my internship, I worked on the Monolith fitness app, a React/Vite-based workout-tracking application focused on simple workout logging, routine creation, and a minimal user experience. My main contribution was developing and testing the early framework for an AI-assisted routine generation feature.

The goal of this feature is to let users create workout routines in two ways:

1. **Manual Routine Creation:** users can create and name their own routine and add exercises manually.
2. **AI Routine Creation:** users can enter a prompt describing their workout goals, time available, body focus, sport background, soreness, experience level, and body weight, then receive a generated workout routine.

The latter feature is still in beta, especially as the Gemini architecture has just been developed.

2. Work Completed

Local App Setup and Testing

I set up the Monolith repository locally, tested the app using PowerShell, and debugged issues related to Node.js, npm, Vite, stale build files, and cached browser versions. I also tested the app on local development servers using both `localhost` and `127.0.0.1`.

The most reliable local run command for the AI branch is:

```
npx concurrently "node server/openaiRoutineServer.mjs" "vite --host 127.0.0.1 --port 5174 --force"
```

Then open:

`http://127.0.0.1:5174`

This avoids some caching issues that occurred with `localhost:5173`.

Routine Creation UI

I helped revise the Routines tab so that the routine creation flow is clearer. Instead of only having a generic “Add Routine” button, the app now supports two clearer paths:

- **Manual**
- **AI Routine**

The AI routine modal allows the user to enter a prompt, workout duration, body weight, sex, and experience level.

AI Routine Generator

I worked on the initial routine-generation logic using a local workout dictionary. The local version uses exercise-to-muscle mappings, movement types, equipment categories, and bodyweight-based starting-weight formulas to generate basic routines without requiring an API key.

The local agent works reasonably well for common prompts, such as:

- “Make me a leg day”
- “Create a chest and triceps workout”
- “Make a 30-minute core workout”

However, it is less accurate for nuanced prompts, such as:

- “Create a chest and back workout”
- “Make me a swimmer upper-body recovery workout”
- “Build a routine for rowing with upper-body focus”

This limitation led to the decision to reintroduce hosted AI providers.

Hosted AI Provider Support

The current AI branch includes work toward a provider-based generation system. The idea is that the AI routine modal can support multiple generation modes:

- **OpenAI**
- **Gemini**

- **Local**

The intended flow is:

User prompt

- Agent mode selected
- If OpenAI/Gemini: request sent to backend server
- If Local: browser uses local dictionary logic
- Routine preview generated
- User adds routine to app

The hosted AI provider logic is handled through the backend server file:

`server/openaiRoutineServer.mjs`

The frontend client file is:

`src/app/lib/openaiRoutineAgentClient.ts`

The local dictionary agent is mainly handled in:

`src/app/lib/routineAgent.ts`

The main UI component is:

`src/app/components/routines/Routines.tsx`

3. Current Status

The AI selector UI now appears correctly when the app is run from source on a clean development URL such as:

`http://127.0.0.1:5174`

Previously, the app appeared to show the old modal because the browser was serving cached/stale UI from an older build. This was not a logic issue in the agent files. Clearing Vite cache, rebuilding `dist`, and using a forced Vite port helped confirm that the updated source code works.

Current visible agent options:

- OpenAI

- Gemini
- Local

OpenAI mode currently depends on having a valid OpenAI API key with available credits. If the account/project has no credits or quota, the backend will return an error such as:

429: You exceeded your current quota

This is not a frontend bug. It means the key or OpenAI project billing needs to be updated.

Gemini mode requires a Gemini API key to be configured. Local mode does not require an API key.

4. Environment Variables

Do not hardcode API keys into frontend React files. The app should use backend environment variables.

For OpenAI:

```
setx OPENAI_API_KEY "your_openai_key_here"
```

For Gemini:

```
setx GEMINI_API_KEY "your_gemini_key_here"
```

After setting an environment variable, close and reopen PowerShell before running the app again.

Important: real `.env` files and API keys should not be committed to GitHub.

5. Known Issues and Notes

1. Browser Cache / Stale Build Issue

At one point, the source code had the correct agent selector, but the browser still showed the old modal. This was caused by stale cached UI or old build artifacts.

Recommended fix:

```
taskkill /F /IM node.exe
```

```
Remove-Item -Recurse -Force .\node_modules\.vite -ErrorAction SilentlyContinue
```

```
Remove-Item -Recurse -Force .\dist -ErrorAction SilentlyContinue
npm run build
npx concurrently "node server/openaiRoutineServer.mjs" "vite --host 127.0.0.1 --port 5174 --force"
```

Then open:

<http://127.0.0.1:5174>

2. Modal Layout

The current AI modal works, but the layout is slightly cramped. On some screen sizes, a horizontal scrollbar may appear. This should be cleaned up by adjusting modal width, button wrapping, and internal spacing.

3. Local Agent Accuracy

The local agent is useful as an offline fallback, but it is not accurate enough for nuanced prompts. It can struggle with prompts that require balancing multiple body groups or understanding sport-specific training needs. Hosted AI should be used for more complex requests.

4. API Provider Errors

If OpenAI or Gemini mode fails, check:

- Is the backend server running?
- Is the correct API key set?
- Does the key have available quota/credits?
- Is the frontend calling the correct backend URL?
- Is the app being run with `dev:ai` or the equivalent command that starts both frontend and backend?

6. Recommended Next Steps

Product Improvements

1. Clean up the AI routine modal layout so the provider buttons and form fields fit without horizontal scrolling.
2. Improve the generated routine preview design so exercises, sets, reps, and weights are easier to scan.

3. Add clearer error messages for API failures, such as “OpenAI quota exceeded” or “Gemini key missing.”
4. Add a loading state when the hosted AI provider is generating a routine.
5. Make the Add Routine button more visually active once a valid routine preview is generated.

AI Improvements

1. Continue using hosted AI providers for nuanced prompts.
2. Keep the local dictionary agent as a fallback, not the primary generation method.
3. Standardize the output schema so OpenAI, Gemini, and Local all return the same routine format.
4. Add stricter validation so generated routines always include:
 - routine name
 - exercise names
 - sets
 - reps
 - suggested weight if possible
 - coach notes or assumptions
5. Improve prompt instructions to ensure balanced routines. For example, if a user requests “chest and back,” the model should include both chest pressing and back pulling movements.

Testing Improvements

Before merging new AI changes, test:

- Manual routine creation
- Local routine generation
- OpenAI routine generation
- Gemini routine generation
- Vague prompt clarification
- Sport-specific prompts
- Multi-muscle prompts like “chest and back”
- API failure cases
- Refresh/cache behavior
- Add Routine behavior after preview generation

Recommended test prompts:

Make me a 1 hour chest and back workout. I want both pressing and pulling movements.

Make me a 45 minute upper body workout for a distance swimmer. I am sore from practice yesterday, so keep it lighter.

Create a 30 minute core workout for a sprinter.

Make me a workout.

Final Reflection

This internship gave me experience working on a real user-facing product that combines frontend development, AI feature design, product testing, and early go-to-market thinking. I learned how to work through practical development issues, including branch management, local setup, cached builds, API keys, provider errors, and UI iteration.

The most valuable part of the project was seeing how an AI feature has to be more than just a model call. The user interface, fallback behavior, prompt structure, data schema, API setup, and error handling all matter for making the feature feel reliable. Going forward, I think the biggest opportunity is to keep improving the AI routine flow so it feels simple, accurate, and useful for real users.

I am very thankful for this opportunity and the skills that I learned and strengthened in these projects will always be on call for my future endeavors.

Please feel free to reach out if you still have any concerns or issues.

<https://www.linkedin.com/in/lucas-li-uchicago/>

lucaszli2025@gmail.com